

Pilha e Fila com alocação encadeada

Estruturas de Dados e Algoritmos

Prof. Sérgio Gorender

Inserção em Pilha

Utiliza uma variável ponteiro, chamada **topo**.

Não precisa utilizar de um nó **header** (cabeçalho)!

function Empilhar(novo-valor, topo)

 new(pt)

 pt.info = novo-valor

 pt.prox = topo

 topo = pt

 return pt

Remoção em Pilha

```
function Desempilhar(topo, valor-recuperado)
    if topo  $\neq \lambda$  then
        pt = topo
        topo = topo.prox
        valor-recuperado = pt.info
        free(pt)
    else underflow
    return topo
```

- Pilha em Python

```
class Elemento:
```

```
    def __init__(self):
```

```
        self.valor = 0
```

```
        self.prox = None
```

```
topo = None
```

```
def empilhar(chave, topo):
```

```
    novo = Elemento()
```

```
    novo.valor = chave
```

```
    novo.prox = topo
```

```
    topo = novo
```

```
    return topo
```

```
def desempilhar(topo):
```

```
    if (topo != None):
```

```
        print(topo.valor)
```

```
        ptaux = topo
```

```
        topo = topo.prox
```

```
        del ptaux
```

```
    else:
```

```
        print("Pilha vazia")
```

```
    return topo
```

Inserção em fila

Utiliza duas variáveis do tipo ponteiro, **inicio** e **fim**.

function Enfileirar(inicio, fim, novo-valor)

 new(pt)

 pt.info = novo-valor

 pt.prox = λ

if fim $\neq \lambda$ **then**

 fim.prox = pt

else

 inicio = pt

 fim = pt

 return inicio, fim

Remoção em fila

function Desenfileirar(inicio, fim, valor-recuperado)

if inicio $\neq \lambda$ **then**

 pt = inicio

 inicio = inicio.prox

 se inicio == λ então

 fim = λ

 valor-recuperado = pt.info

 free(pt)

else underflow

 return inicio, fim, valor-recuperado

- Fila em Python

```
class Elemento:  
    def __init__(self):  
        self.valor = 0  
        self.prox = None
```

```
class Ponteiros:  
    def __init__(self):  
        self.inicio = None  
        self.fim = None
```

```
ponteiros = Ponteiros()  
ponteiros.fim = None  
ponteiros.inicio = None
```

```
def enfileirar(chave, ponteiros):  
    novo = Elemento()  
    novo.valor = chave  
    novo.prox = None  
    if (ponteiros.fim != None):  
        ponteiros.fim.prox = novo  
    else:  
        ponteiros.inicio = novo  
    ponteiros.fim = novo  
    return ponteiros
```

- Fila em Python

```
class Elemento:  
    def __init__(self):  
        self.valor = 0  
        self.prox = None
```

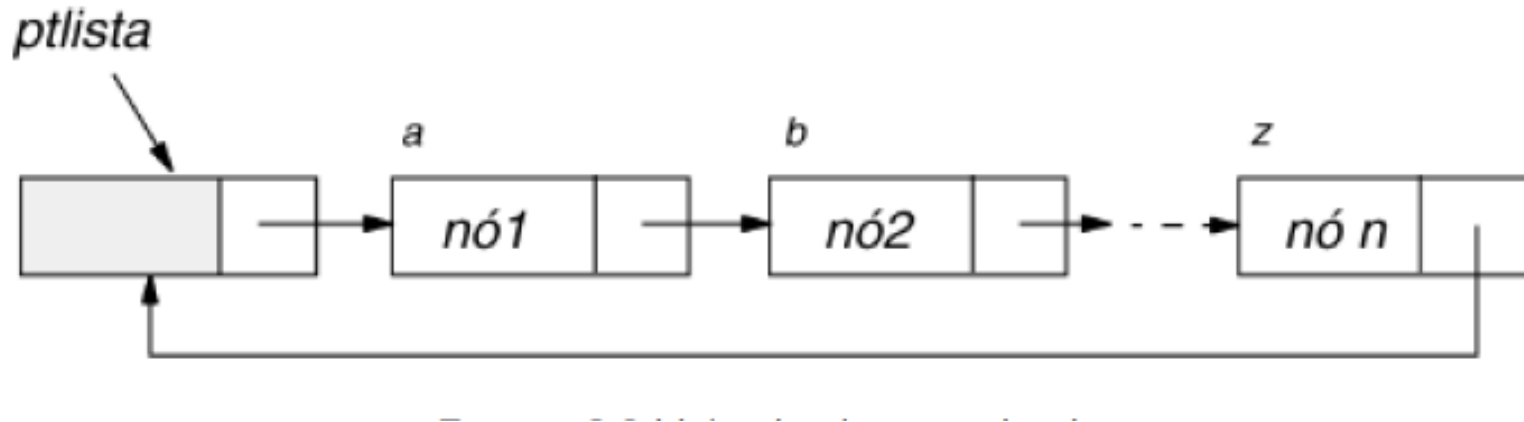
```
class Ponteiros:  
    def __init__(self):  
        self.inicio = None  
        self.fim = None
```

```
ponteiros = Ponteiros()  
ponteiros.fim = None  
ponteiros.inicio = None
```

```
def desenfileirar(ponteiros):  
    if (ponteiros.inicio != None):  
        print(ponteiros.inicio.valor)  
        ptaux = ponteiros.inicio  
        ponteiros.inicio = ponteiros.inicio.prox  
        del ptaux  
    if (ponteiros.inicio == None):  
        ponteiros.fim = None  
    else:  
        print("Fila vazia")  
    return ponteiros
```


Listas circulares

- Lista circular com nó *head* (nó inicial sem valor armazenado).
- Último nó aponta para o *head*.



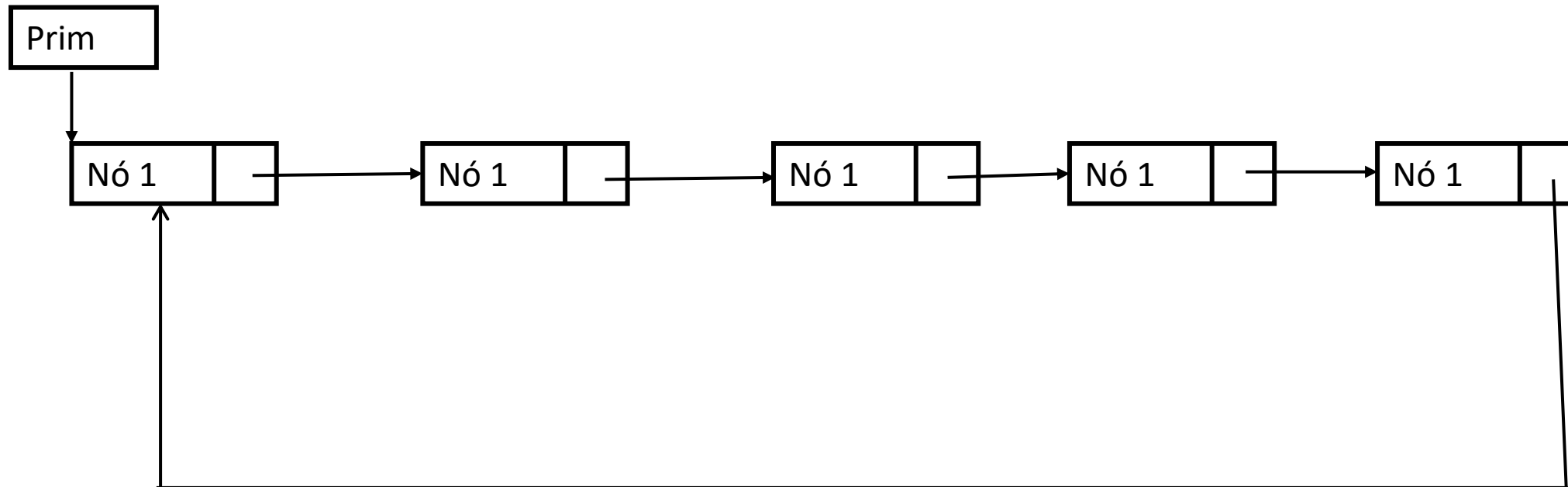
Busca em lista circular encadeada e ordenada com nó header

```
function busca-cir(x, ant, pont)
    ant = ptlista
    pont = ptlista.prox
    while pont  $\neq$  ptlista and pont.chave < x do
        ant = pont
        pont = pont.prox
    if pont.chave == x then
        result = pont -- “chave localizada”
    else
        result =  $\lambda$  -- “chave não localizada”
    return ant, result
```

Busca em lista circular encadeada e ordenada com nó header - Python

```
def busca(chave):  
    global primeiro  
    ponteiros = Ponteiros()  
    ponteiros.pont = primeiro.prox  
    ponteiros.ant = primeiro  
    while ((ponteiros.pont.valor != 0 ) and  
(ponteiros.pont.valor < chave)):  
        ponteiros.ant = ponteiros.pont  
        ponteiros.pont = ponteiros.pont.prox  
    return ponteiros
```

Listas Circulares sem *head*



Busca em lista circular encadeada e ordenada sem nó header

```
function busca-cir(x, ant, pont)
    ant =  $\lambda$ 
    pont = prim
    while (pont.chave < x) and (ant =  $\lambda$  or ant.prox  $\neq$  prim) do
        ant = pont
        pont = pont.prox
    if pont.chave == x then
        “chave localizada”
    senão “chave não localizada”
```